

ATENA NA PRÁTICA

Programação completa de uma IA nativa para Linux

Do Linux Mint e C17 ao Core, providers, RAG, agentes, tools, interface, pacote .deb e integração ao Athenas OS

Edição prática revisada 1.0 - agosto de 2026

Linux Mint | Debian/Ubuntu family | amd64 primeiro | ARM64/RISC-V depois

Objetivo — Este livro não é uma coleção de exemplos soltos. Ele conduz a construção do Atena como projeto real, com checkpoints compiláveis, diagnóstico de erros em cada marco, segurança por projeto e caminhos claros para personalização.

Como usar este livro

A abordagem é a mesma de uma mentoria técnica em chat: fazemos uma pequena parte, entendemos o que ela representa, compilamos, testamos, diagnosticamos, registramos no Git e só então avançamos. Não existe “copiar 3 mil linhas e rezar”. Cada marco cria algo executável.

- Ambiente principal: Linux Mint, com Geany como editor leve e terminal como ferramenta de build e diagnóstico.
- Linguagem do Core: C17. C++ aparece apenas onde bibliotecas externas exigirem ou onde o projeto decidir conscientemente.
- Build: CMake. Testes: CTest + assert no início, evoluindo para sanitizers, integração e fuzzing.
- IA local: llama.cpp/llama-server como runtime de referência; Ollama e APIs remotas entram como providers substituíveis.
- Persistência: SQLite. RAG leve: FTS5. Transporte HTTP: libcurl. JSON: json-c.
- Regra de segurança: texto gerado pelo modelo nunca vira shell arbitrário. Toda ação passa por Tool Registry, schema, política, whitelist e confirmação quando necessário.

Correção importante da auditoria — Algumas críticas ao PDF anterior estavam certas; outras precisavam de refinamento. Exemplo: ProtectHome=read-only não impede leitura do HOME, apenas escrita. O problema real é que ele permite leitura ampla demais. Nesta versão usamos permissões granulares e política de arquivos aprovados. Também corrigimos a validação de endpoint com parser de URL, pois comparar strings com dois strncmp negados não é uma validação correta.

Mapa completo do projeto

O produto final desta edição é uma Atena avançada o suficiente para ser usada como base real do Athenas OS. O livro divide o trabalho em marcos verificáveis:

Marco	Entrega prática	Critério de aceite
M0	Contratos, CLI, provider mock, testes	Compila sem warnings críticos e requests atravessam o pipeline.
M1	Policy + SQLite + auditoria	Dados persistem e decisões de privacidade têm testes.
M2	llama-server + provider HTTP + streaming/cancelamento	Pergunta real recebe tokens de modelo GGUF local.
M3	Model Manager + Hardware Profiler + Router	Modelo é validado contra RAM/disco/capacidade e escolhido conscientemente.
M4	RAG FTS5 + citações	PDF/Markdown textual vira evidência recuperável com locator.
M5	Tool Engine + Agent Engine	LLM propõe; Core valida; usuário confirma; tool executa.
M6	systemd user + IPC + UI	CLI/UI usam o mesmo Core sem bloquear a interface.
M7	Pacote .deb + ISO Athenas	Instala, atualiza, remove e entra na build da distro.
M8	Hardening, CI, ARM64/RISC-V	Matriz de testes, sanitizers, releases e portabilidade.

Parte 1 - Preparando o Linux Mint

1.1 O que instalar

BASH

```
sudo apt update
sudo apt install -y \
  build-essential cmake ninja-build git gdb pkg-config geany geany-plugins \
  libjson-c-dev libsqlite3-dev libcurl4-openssl-dev \
  libssl-dev ca-certificates curl wget jq sqlite3 \
  valgrind clang clang-tidy cppcheck
```

Você não “instala C”. Você instala um compilador e o ecossistema de desenvolvimento. build-essential traz GCC, Make, headers da libc e ferramentas básicas. CMake descreve como os arquivos viram bibliotecas e executáveis. Geany é apenas o editor, e isso é suficiente.

1.2 Diagnóstico do ambiente

BASH

```
cat /etc/os-release
uname -m
free -h
gcc --version
cmake --version
git --version
pkg-config --modversion json-c
pkg-config --modversion sqlite3
pkg-config --modversion libcurl
```

Se “package not found” aparecer — Verifique se sudo apt update terminou sem erro. Em Linux Mint, confira também se os repositórios oficiais estão ativos em Gerenciador de Aplicativos > Fontes de software.

1.3 Teste C mínimo

C

```
#include <stdio.h>

int main(void) {
    puts("Ambiente da Atena funcionando");
    return 0;
}
```

BASH

```
gcc -std=c17 -Wall -Wextra -Wpedantic teste.c -o teste
./teste
```

Erro	Causa provável	Como diagnosticar
gcc: command not found	build-essential ausente	which gcc; dpkg -l build-essential
fatal error: stdio.h	headers da libc ausentes/corrompidos	dpkg -L libc6-dev grep stdio.h
Permission denied ao executar	arquivo em filesystem noexec ou permissão	ls -l ./teste; mount grep noexec
Segmentation fault	erro no programa, não no compilador	gdb ./teste; run; bt

Parte 2 - Repositório e arquitetura física

BASH

```
mkdir -p ~/Projetos/atena
cd ~/Projetos/atena
git init
mkdir -p include/atena core providers cli tests schemas memory rag tools agent model hardware ipc ui
packaging/debian docs/adr
```

O repositório separa contratos públicos, implementação interna, adapters externos e integração com o sistema. A separação é um mecanismo de segurança: o Core não deve conhecer detalhes de cada provider, e um provider não deve ganhar permissão para alterar política.

Diretório	Responsabilidade
include/atena	API pública em C: tipos, contratos e funções expostas.
core	Request Engine, política, router, auditoria e coordenação.
providers	Adapters para mock, llama-server, Ollama e APIs compatíveis.
memory	SQLite, conversas, preferências e memória aprovada.
rag	extração, chunking, FTS5, locators e citações.
tools	registro de ferramentas, schemas e implementações.
agent	planejamento e ciclo proposta->validação->execução.
model	catálogo, downloads, hash, benchmark, remoção.
hardware	CPU, RAM, disco, GPU/backend e perfil de capacidade.
ipc	socket Unix/D-Bus para UI/CLI falarem com o Core.
ui	interface gráfica, sem lógica de segurança duplicada.
tests	unidade, integração, segurança, regressão.
packaging/debian	pacotes .deb, units, desktop files e políticas.

Parte 3 - M0: contratos que compilam

Antes do modelo, construímos a linguagem interna da Atena. Os nomes abaixo serão usados por providers, CLI, UI e testes.

3.1 status.h e status.c

C

```
#ifndef ATENA_STATUS_H
#define ATENA_STATUS_H

typedef enum {
    ATENA_OK = 0,
    ATENA_ERR_INVALID,
    ATENA_ERR_NOMEM,
    ATENA_ERR_IO,
    ATENA_ERR_POLICY,
    ATENA_ERR_PROVIDER,
    ATENA_ERR_CANCELLED,
    ATENA_ERR_TIMEOUT,
    ATENA_ERR_UNSUPPORTED
} AtenaStatus;

const char *atena_status_string(AtenaStatus status);
#endif
```

C

```
#include "atena/status.h"

const char *atena_status_string(AtenaStatus s) {
    switch (s) {
        case ATENA_OK: return "ok";
        case ATENA_ERR_INVALID: return "invalid argument";
        case ATENA_ERR_NOMEM: return "out of memory";
        case ATENA_ERR_IO: return "I/O error";
        case ATENA_ERR_POLICY: return "blocked by policy";
        case ATENA_ERR_PROVIDER: return "provider error";
        case ATENA_ERR_CANCELLED: return "cancelled";
        case ATENA_ERR_TIMEOUT: return "timeout";
        case ATENA_ERR_UNSUPPORTED: return "unsupported";
    }
    return "unknown error";
}
```

Erro corrigido — Use ATENA_ERR_INVALID com underscore. “ATENA ERR_INVALID” é sintaxe inválida em C.

3.2 request.h: segurança vira dado

C

```
#ifndef ATENA_REQUEST_H
#define ATENA_REQUEST_H
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "atena/status.h"

typedef enum {
    ATENA_PRIVACY_PUBLIC = 0,
    ATENA_PRIVACY_NORMAL,
    ATENA_PRIVACY_SENSITIVE,
    ATENA_PRIVACY_PRIVATE,
    ATENA_PRIVACY_SECRET
} AtenaPrivacyClass;

#define ATENA_MAX_ALLOWED_TOOLS 16

typedef struct {
    uint64_t id;
    const char *session_id;
    const char *user_text;
    AtenaPrivacyClass privacy;
    bool allow_network;
    bool allow_tools;
    size_t allowed_tool_count;
    const char *allowed_tools[ATENA_MAX_ALLOWED_TOOLS];
} AtenaRequest;
```

```

AtenaStatus atena_request_validate(const AtenaRequest *r);
bool atena_request_tool_allowed(const AtenaRequest *r, const char *tool_name);
#endif

```

C

```

#include "atena/request.h"
#include <string.h>

#define ATENA_MAX_USER_TEXT_BYTES (64u * 1024u)

AtenaStatus atena_request_validate(const AtenaRequest *r) {
    if (!r || !r->session_id || !r->user_text) return ATENA_ERR_INVALID;
    if (r->session_id[0] == '\0' || r->user_text[0] == '\0') return ATENA_ERR_INVALID;
    if (strlen(r->user_text) > ATENA_MAX_USER_TEXT_BYTES) return ATENA_ERR_INVALID;
    if (r->privacy < ATENA_PRIVACY_PUBLIC || r->privacy > ATENA_PRIVACY_SECRET)
        return ATENA_ERR_INVALID;
    if (r->allowed_tool_count > ATENA_MAX_ALLOWED_TOOLS) return ATENA_ERR_INVALID;
    return ATENA_OK;
}

bool atena_request_tool_allowed(const AtenaRequest *r, const char *name) {
    if (!r || !name || !r->allow_tools) return false;
    for (size_t i = 0; i < r->allowed_tool_count; ++i) {
        if (r->allowed_tools[i] && strcmp(r->allowed_tools[i], name) == 0) return true;
    }
    return false;
}

```

Falha arquitetural corrigida — A whitelist de tools agora é parte explícita do request. O Agent Engine não pode executar uma tool apenas porque o JSON tem schema válido.

Parte 4 - Eventos, provider API e provider mock

4.1 event.h

C

```
#ifndef ATENA_EVENT_H
#define ATENA_EVENT_H
#include <stdint.h>
#include "atena/status.h"

typedef enum {
    ATENA_EVENT_TEXT_DELTA = 0,
    ATENA_EVENT_TOOL_PROPOSAL,
    ATENA_EVENT_CITATION,
    ATENA_EVENT_USAGE,
    ATENA_EVENT_DONE,
    ATENA_EVENT_ERROR
} AtenaEventType;

typedef struct {
    AtenaEventType type;
    uint64_t request_id;
    const char *json_payload;
} AtenaEvent;

typedef AtenaStatus (*AtenaEventCallback)(const AtenaEvent *, void *user_data);
#endif
```

4.2 provider.h

C

```
#ifndef ATENA_PROVIDER_H
#define ATENA_PROVIDER_H
#include <stdint.h>
#include "atena/event.h"
#include "atena/request.h"

typedef struct AtenaProvider AtenaProvider;

typedef struct {
    AtenaStatus (*start)(AtenaProvider *self);
    AtenaStatus (*generate)(AtenaProvider *self, const AtenaRequest *request,
                           AtenaEventCallback cb, void *user_data);
    void (*cancel)(AtenaProvider *self, uint64_t request_id);
    void (*destroy)(AtenaProvider *self);
} AtenaProviderVTable;

struct AtenaProvider {
    const AtenaProviderVTable *ops;
    const char *id;
    const char *display_name;
    void *impl;
};
#endif
```

Erro corrigido — O tipo chama-se AtenaProviderVTable em todos os pontos. Não misture AtenaProviderTable e AtenaProviderVTable.

4.3 Mock provider correto

C

```
#include "atena/provider.h"
#include <stdlib.h>

static AtenaStatus emit(AtenaEventCallback cb, void *ud, AtenaEventType type,
                        uint64_t id, const char *payload) {
    if (!cb) return ATENA_ERR_INVALID;
    AtenaEvent e = {.type=type, .request_id=id, .json_payload=payload};
    return cb(&e, ud);
}

static AtenaStatus mock_start(AtenaProvider *self) {
    return self ? ATENA_OK : ATENA_ERR_INVALID;
}
```

```

static AtenaStatus mock_generate(AtenaProvider *self, const AtenaRequest *r,
                                AtenaEventCallback cb, void *ud) {
    if (!self || !r || !cb) return ATENA_ERR_INVALID;
    AtenaStatus s = atena_request_validate(r);
    if (s != ATENA_OK) return s;
    s = emit(cb, ud, ATENA_EVENT_TEXT_DELTA, r->id,
            "{\"text\": \"Atena mock: recebi sua solicitacao.\"}");
    if (s != ATENA_OK) return s;
    return emit(cb, ud, ATENA_EVENT_DONE, r->id, "{\"status\": \"ok\"}");
}

static void mock_cancel(AtenaProvider *self, uint64_t request_id) {
    (void)self; (void)request_id;
}

static void mock_destroy(AtenaProvider *self) { free(self); }

static const AtenaProviderVTable MOCK_OPS = {
    .start=mock_start, .generate=mock_generate,
    .cancel=mock_cancel, .destroy=mock_destroy
};

AtenaStatus atena_mock_provider_create(AtenaProvider **out) {
    if (!out) return ATENA_ERR_INVALID;
    AtenaProvider *p = calloc(1, sizeof(*p));
    if (!p) return ATENA_ERR_NOMEM;
    p->ops=&MOCK_OPS; p->id="mock.local"; p->display_name="Atena Mock";
    *out=p;
    return ATENA_OK;
}

```

Erro corrigido — A string JSON correta em C precisa escapar aspas: `{\"text\": \"...\"}`. O PDF anterior tinha uma sequência inválida.

Parte 5 - CMake, CLI e primeira compilação real

CMAKE

```
cmake_minimum_required(VERSION 3.25)
project(atenas VERSION 0.1.0 LANGUAGES C)
set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
include(CTest)
find_package(PkgConfig REQUIRED)
pkg_check_modules(JSONC REQUIRED IMPORTED_TARGET json-c)
pkg_check_modules(SQLITE REQUIRED IMPORTED_TARGET sqlite3)
pkg_check_modules(CURL REQUIRED IMPORTED_TARGET libcurl)

add_library(atenas-core STATIC core/status.c core/request.c)
target_include_directories(atenas-core PUBLIC include)
target_compile_options(atenas-core PRIVATE -Wall -Wextra -Wpedantic -Wconversion -Wshadow)

add_library(atenas-provider-mock STATIC providers/mock.c)
target_include_directories(atenas-provider-mock PUBLIC include)
target_link_libraries(atenas-provider-mock PUBLIC atenas-core)

add_executable(atenas-cli cli/main.c)
target_link_libraries(atenas-cli PRIVATE atenas-core atenas-provider-mock)

if(BUILD_TESTING)
    add_subdirectory(tests)
endif()
```

BASH

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure
./build/atenas-cli "Ola Atenas"
```

5.1 Como diagnosticar esta compilação

Mensagem	Significado	Ação
fatal error: atenas/xxx.h: No such file	include path não chegou ao target	confira target_include_directories e nome do arquivo
undefined reference to ...	header foi visto, mas implementação não foi linkada	adicione o .c ao add_library ou target_link_libraries
unknown type name AtenasProviderVTable	typedef ausente ou nome divergente	grep -R "AtenasProviderVTable" include core providers
multiple definition of ...	função global definida em dois .c ou corpo no .h	deixe declaração no header e implementação em um único .c
warning treated as error	-Werror em CI	corrija o warning; não esconda com cast sem entender
ctest: No tests were found	subdiretório tests não chamou add_test	confira include(CTest), BUILD_TESTING e tests/CMakeLists.txt

BASH

```
cmake --build build --verbose
nm -C build/libatenas-core.a | less
ldd build/atenas-cli
file build/atenas-cli
```

Parte 6 - Testes, sanitizers e método de diagnóstico

Cada marco terá três níveis: compilador, testes e execução controlada. O debugger entra quando o erro não é evidente.

BASH

```
cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug \  
-DCMAKE_C_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g"  
cmake --build build-asan -j2  
ctest --test-dir build-asan --output-on-failure
```

BASH

```
gdb --args ./build/atena-cli "teste"  
# dentro do gdb:  
run  
bt  
frame 0  
info locals  
print variavel
```

Sintoma	Ferramenta	Pergunta a responder
crash/segfault	ASan + GDB	qual linha acessou memória inválida?
leak	ASan/Valgrind	quem alocou e não liberou?
resultado errado	teste unitário + breakpoint	qual hipótese do código está falsa?
travamento	GDB attach + bt all	qual thread está bloqueada?
erro de link	nm/readelf/ldd	o símbolo existe e está na biblioteca correta?
erro intermitente	logs estruturados + teste repetido	estado global ou condição de corrida?

Parte 7 - Policy Engine: privacidade antes de qualidade

C

```
#ifndef ATENA_POLICY_H
#define ATENA_POLICY_H
#include <stdbool.h>
#include "atena/request.h"

typedef struct {
    bool may_use_network;
    bool may_use_remote_provider;
    bool may_use_tools;
} AtenaPolicyDecision;

AtenaStatus atena_policy_evaluate(const AtenaRequest *r, AtenaPolicyDecision *out);
#endif
```

C

```
#include "atena/policy.h"

AtenaStatus atena_policy_evaluate(const AtenaRequest *r, AtenaPolicyDecision *out) {
    if (!r || !out) return ATENA_ERR_INVALID;
    out->may_use_network = r->allow_network;
    out->may_use_tools = r->allow_tools;
    switch (r->privacy) {
        case ATENA_PRIVACY_PUBLIC:
        case ATENA_PRIVACY_NORMAL:
            out->may_use_remote_provider = r->allow_network;
            break;
        case ATENA_PRIVACY_SENSITIVE:
        case ATENA_PRIVACY_PRIVATE:
        case ATENA_PRIVACY_SECRET:
            out->may_use_remote_provider = false;
            break;
        default:
            return ATENA_ERR_INVALID;
    }
    return ATENA_OK;
}
```

Regra — Falha fechada: um valor desconhecido de privacidade não libera a rede. Política vem antes do router de qualidade/custo.

Parte 8 - SQLite: memória não é uma caixa única

8.1 Schema corrigido SQL

```
PRAGMA foreign_keys = ON;
CREATE TABLE IF NOT EXISTS schema_migrations(
  version INTEGER PRIMARY KEY,
  applied_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS conversations(
  id TEXT PRIMARY KEY,
  title TEXT,
  created_at TEXT NOT NULL,
  privacy_class TEXT NOT NULL,
  retention_policy TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS messages(
  id TEXT PRIMARY KEY,
  conversation_id TEXT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  role TEXT NOT NULL,
  content TEXT NOT NULL,
  provider_id TEXT,
  model_id TEXT,
  created_at TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_messages_conversation ON messages(conversation_id);
```

Erro corrigido — conversation_id é um identificador único sem espaço. “conversation id” não representa o nome pretendido da coluna.

8.2 Prepared statements C

```
const char *sql = "INSERT INTO messages(id,conversation_id,role,content,created_at)
VALUES(?,?,?,datetime('now'))";
sqlite3_stmt *st = NULL;
if (sqlite3_prepare_v2(db, sql, -1, &st, NULL) != SQLITE_OK) return ATENA_ERR_IO;
sqlite3_bind_text(st, 1, id, -1, SQLITE_TRANSIENT);
sqlite3_bind_text(st, 2, conversation_id, -1, SQLITE_TRANSIENT);
sqlite3_bind_text(st, 3, role, -1, SQLITE_TRANSIENT);
sqlite3_bind_text(st, 4, content, -1, SQLITE_TRANSIENT);
int rc = sqlite3_step(st);
sqlite3_finalize(st);
return rc == SQLITE_DONE ? ATENA_OK : ATENA_ERR_IO;
```

8.3 Diagnóstico SQLite BASH

```
sqlite3 ~/.local/share/atenas/atenas.db ".tables"
sqlite3 ~/.local/share/atenas/atenas.db "PRAGMA integrity_check;"
sqlite3 ~/.local/share/atenas/atenas.db "PRAGMA foreign_keys;"
```

Parte 9 - Construindo e executando llama.cpp no Mint

O runtime local de referência é o llama.cpp. A interface exata pode evoluir, então o livro ensina a verificar a versão instalada em vez de presumir uma flag para sempre.

BASH

```
sudo apt install -y git cmake build-essential
cd ~/Projetos
git clone https://github.com/ggml-org/llama.cpp.git
cd llama.cpp
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release -t llama-server -j2
./build/bin/llama-server --help | less
```

Com um GGUF já obtido de fonte confiável:

BASH

```
./build/bin/llama-server -m /caminho/modelo.gguf -c 2048 --host 127.0.0.1 --port 18080
```

BASH

```
curl -s http://127.0.0.1:18080/health | jq .
curl -s http://127.0.0.1:18080/v1/models | jq .
```

Compatibilidade de versão — Em versões recentes do ecossistema llama.cpp também pode existir uma CLI consolidada “llama serve”. Sempre rode --help na versão que você compilou e mantenha o adapter HTTP da Atena isolado para absorver mudanças externas.

Parte 10 - Endpoint seguro: não confie em strings

O provider local deve aceitar apenas loopback quando configurado como LOOPBACK_ONLY. Não use testes manuais frágeis de prefixo. Parseie a URL com a API de URL da libcurl e verifique scheme, host, porta e ausência de credenciais.

C

```
#include <curl/curl.h>
#include <string.h>

static bool is_loopback_host(const char *host) {
    return host && (
        strcmp(host, "127.0.0.1") == 0 ||
        strcmp(host, "localhost") == 0 ||
        strcmp(host, "::1") == 0
    );
}

AtenaStatus atena_validate_loopback_endpoint(const char *endpoint) {
    if (!endpoint) return ATENA_ERR_INVALID;
    CURLU *u = curl_url();
    if (!u) return ATENA_ERR_NOMEM;
    CURLUcode rc = curl_url_set(u, CURLUPART_URL, endpoint, 0);
    if (rc != CURLUE_OK) { curl_url_cleanup(u); return ATENA_ERR_INVALID; }

    char *scheme=NULL, *host=NULL, *user=NULL, *password=NULL;
    rc = curl_url_get(u, CURLUPART_SCHEME, &scheme, 0);
    if (rc != CURLUE_OK || strcmp(scheme, "http") != 0) goto invalid;
    rc = curl_url_get(u, CURLUPART_HOST, &host, 0);
    if (rc != CURLUE_OK || !is_loopback_host(host)) goto invalid;
    if (curl_url_get(u, CURLUPART_USER, &user, 0) == CURLUE_OK) goto invalid;
    if (curl_url_get(u, CURLUPART_PASSWORD, &password, 0) == CURLUE_OK) goto invalid;

    curl_free(scheme); curl_free(host); curl_url_cleanup(u);
    return ATENA_OK;
invalid:
    curl_free(scheme); curl_free(host); curl_free(user); curl_free(password);
    curl_url_cleanup(u);
    return ATENA_ERR_INVALID;
}
```

Por que a correção sugerida na auditoria não serve — A condição com dois !strcmp unidos por && exigiria que a mesma string começasse simultaneamente com “127.0.0.1” e “localhost”. Além disso, prefixo não valida URL, porta, esquema nem userinfo.

Parte 11 - HTTP comum, timeout, streaming e cancelamento

O provider pode usar libcurl, mas o design precisa nascer cancelável. `CURLOPT_CONNECTTIMEOUT_MS` e `CURLOPT_TIMEOUT_MS` possuem underscores e recebem milissegundos. O callback de progresso pode abortar a transferência; o callback de escrita recebe chunks incrementais.

C

```
typedef struct {
    volatile int cancelled;
    AtenaEventCallback cb;
    void *user_data;
    uint64_t request_id;
} HttpRequestCtx;

static int xferinfo(void *clientp, curl_off_t dltotal, curl_off_t dlnow,
                   curl_off_t ultotal, curl_off_t ulnow) {
    (void)dltotal; (void)dlnow; (void)ultotal; (void)ulnow;
    HttpRequestCtx *ctx = clientp;
    return (ctx && ctx->cancelled) ? 1 : 0; /* CURLE_ABORTED_BY_CALLBACK */
}

static void configure_common(CURL *curl, HttpRequestCtx *ctx) {
    curl_easy_setopt(curl, CURLOPT_CONNECTTIMEOUT_MS, 3000L);
    curl_easy_setopt(curl, CURLOPT_TIMEOUT_MS, 60000L);
    curl_easy_setopt(curl, CURLOPT_NOSIGNAL, 1L);
    curl_easy_setopt(curl, CURLOPT_XFERINFOFUNCTION, xferinfo);
    curl_easy_setopt(curl, CURLOPT_XFERINFODATA, ctx);
    curl_easy_setopt(curl, CURLOPT_NOPROGRESS, 0L);
    curl_easy_setopt(curl, CURLOPT_MAXREDIRS, 0L);
}
```

Arquitetura da UI — Mesmo uma chamada HTTP cancelável não deve rodar no thread gráfico. A UI fala com atena-core via IPC. O Core executa a requisição em seu event loop/worker e transmite eventos; cancelar marca o request e aborta o transporte.

Parte 12 - JSON robusto com json-c

JSON vindo de modelo ou servidor é entrada não confiável. Use tokenizer com profundidade explícita e modo estrito/UTF-8 quando disponível. Nunca acesse `choices[0]` supondo que ele existe.

C

```
#include <json-c/json.h>
#include <json-c/json_tokener.h>

AtenaStatus parse_reply(const char *buf, size_t len, struct json_object **out) {
    if (!buf || !out || len > (4u * 1024u * 1024u)) return ATENA_ERR_INVALID;
    struct json_tokener *tok = json_tokener_new_ex(32);
    if (!tok) return ATENA_ERR_NOMEM;
#ifdef JSON_TOKENER_STRICT
    json_tokener_set_flags(tok, JSON_TOKENER_STRICT | JSON_TOKENER_VALIDATE_UTF8);
#endif
    struct json_object *root = json_tokener_parse_ex(tok, buf, (int)len);
    enum json_tokener_error e = json_tokener_get_error(tok);
    json_tokener_free(tok);
    if (e != json_tokener_success || !root) return ATENA_ERR_PROVIDER;
    *out = root;
    return ATENA_OK;
}
```

C

```
struct json_object *choices = NULL;
if (!json_object_object_get_ex(root, "choices", &choices) ||
    !json_object_is_type(choices, json_type_array) ||
    json_object_array_length(choices) == 0) {
    json_object_put(root);
    return ATENA_ERR_PROVIDER;
}
```

Erro corrigido — A função é `json_object_new_double(0.3)`, com underscores.

Parte 13 - Provider llama-server: da requisição ao modelo

O adapter transforma AtenaRequest em JSON compatível com o endpoint do runtime e traduz a resposta de volta para AtenaEvent. A identidade da Atena permanece no Core; o provider é transporte/capacidade.

C

```
struct json_object *body = json_object_new_object();
struct json_object *messages = json_object_new_array();
struct json_object *msg = json_object_new_object();
json_object_object_add(msg, "role", json_object_new_string("user"));
json_object_object_add(msg, "content", json_object_new_string(request->user_text));
json_object_array_add(messages, msg);
json_object_object_add(body, "messages", messages);
json_object_object_add(body, "temperature", json_object_new_double(0.3));
json_object_object_add(body, "stream", json_object_new_boolean(1));
```

Para produção, o parser de streaming precisa manter buffer parcial porque um callback HTTP pode cortar JSON/SSE no meio de uma linha. Nunca assuma que um chunk de rede equivale a um objeto lógico completo.

Parte 14 - Model Manager: disco, hash e catálogo

14.1 Espaço livre antes do download

C

```
#include <sys/statvfs.h>
#include <stdint.h>

AtenaStatus ensure_free_space(const char *path, uint64_t download_bytes) {
    struct statvfs s;
    if (!path || statvfs(path, &s) != 0) return ATENA_ERR_IO;
    uint64_t free_bytes = (uint64_t)s.f_bavail * (uint64_t)s.f_frsize;
    if (download_bytes > UINT64_MAX / 2) return ATENA_ERR_INVALID;
    uint64_t required = download_bytes * 2; /* arquivo + margem/cache/temporários */
    return free_bytes >= required ? ATENA_OK : ATENA_ERR_IO;
}
```

O valor 2x é política conservadora, não lei física. O manager pode parametrizar margem absoluta e relativa. O objetivo é falhar antes de preencher o filesystem.

14.2 Fluxo seguro de instalação

1. Obter metadata: tamanho esperado, SHA-256, licença e origem.
2. Verificar espaço com statvfs no filesystem de destino.
3. Baixar para arquivo .part usando HTTPS e limite de tamanho.
4. Calcular SHA-256 local; comparar em tempo constante quando aplicável.
5. Renomear atomicamente .part para .gguf apenas após sucesso.
6. Registrar no SQLite catálogo de modelo, quantização, contexto e caminho.
7. Executar benchmark curto antes de marcar como recomendado.

Parte 15 - Hardware Profiler e compatibilidade

A Atena precisa saber onde está rodando. “2 GB de RAM” não é suficiente: devemos medir memória disponível, swap, arquitetura, threads, disco e, quando possível, GPU/backends.

C

```
#include <sys/sysinfo.h>
#include <unistd.h>

typedef struct {
    unsigned long long ram_total;
    unsigned long long ram_free;
    long cpu_threads;
} AtenaHardwareProfile;

AtenaStatus atena_hw_profile(AtenaHardwareProfile *out) {
    if (!out) return ATENA_ERR_INVALID;
    struct sysinfo si;
    if (sysinfo(&si) != 0) return ATENA_ERR_IO;
    out->ram_total = (unsigned long long)si.totalram * si.mem_unit;
    out->ram_free = (unsigned long long)si.freeram * si.mem_unit;
    out->cpu_threads = sysconf(_SC_NPROCESSORS_ONLN);
    return ATENA_OK;
}
```

Em Linux moderno, “free RAM” isolada é enganosa por causa de page cache. Uma versão avançada lê /proc/meminfo e usa MemAvailable. O benchmark real deve superar estimativas teóricas.

Parte 16 - Provider Router e Model Router

O Provider Router escolhe ambiente/transporte. O Model Router escolhe modelo dentro dele. Eles recebem candidatos já filtrados por privacidade e capacidade.

C

```
typedef struct {
    double quality;
    double latency_ms;
    double cost;
    bool local;
    bool healthy;
} AtenaCandidateScore;

static double score(const AtenaCandidateScore *c) {
    if (!c || !c->healthy) return -1e30;
    return c->quality * 4.0 - c->latency_ms * 0.001 - c->cost * 2.0 + (c->local ? 0.5 : 0.0);
}
```

Não use um score mágico — Pesos devem ser configuráveis, testados e explicáveis. Privacy/Policy são filtros duros e nunca entram apenas como “peso”.

Parte 17 - RAG Lite correto com SQLite FTS5

17.1 Tabelas base + FTS5 external content SQL

```
CREATE TABLE documents(
  id INTEGER PRIMARY KEY,
  path TEXT NOT NULL UNIQUE,
  title TEXT,
  privacy_class TEXT NOT NULL,
  sha256 TEXT NOT NULL
);
CREATE TABLE chunks(
  id INTEGER PRIMARY KEY,
  document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  page_start INTEGER,
  page_end INTEGER,
  heading TEXT,
  text TEXT NOT NULL
);
CREATE VIRTUAL TABLE chunks_fts USING fts5(
  heading, text,
  content='chunks', content_rowid='id',
  tokenize='unicode61'
);
CREATE TRIGGER chunks_ai AFTER INSERT ON chunks BEGIN
  INSERT INTO chunks_fts(rowid, heading, text) VALUES(new.id,new.heading,new.text);
END;
CREATE TRIGGER chunks_ad AFTER DELETE ON chunks BEGIN
  INSERT INTO chunks_fts(chunks_fts,rowid,heading,text) VALUES('delete',old.id,old.heading,old.text);
END;
CREATE TRIGGER chunks_au AFTER UPDATE ON chunks BEGIN
  INSERT INTO chunks_fts(chunks_fts,rowid,heading,text) VALUES('delete',old.id,old.heading,old.text);
  INSERT INTO chunks_fts(rowid,heading,text) VALUES(new.id,new.heading,new.text);
END;
```

Erro corrigido — A tabela base é chunks, não chunks_c. Ao usar external content, declare content='chunks' e mantenha o índice sincronizado por triggers ou comando rebuild.

17.2 Consulta BM25 com locator SQL

```
SELECT c.id, c.document_id, c.page_start, c.page_end, c.heading, c.text,
       bm25(chunks_fts) AS rank
FROM chunks_fts
JOIN chunks AS c ON c.id = chunks_fts.rowid
WHERE chunks_fts MATCH ?
ORDER BY rank
LIMIT ?;
```

Parte 18 - Ingestão: arquivos autorizados, não HOME inteiro

RAG não deve varrer o diretório pessoal silenciosamente. O usuário escolhe arquivos/pastas. O Core registra uma grant explícita e indexa apenas essa origem. Uma estratégia simples é copiar documentos aprovados para o armazenamento da Atena; outra é guardar um locator e validar acesso em cada leitura.

- Detectar tipo por conteúdo/MIME quando possível, não apenas extensão.
- Impor limite de bytes e páginas antes de extrair.
- PDF textual: extrator de texto. PDF escaneado: OCR opcional e confiança marcada.
- Markdown: preservar headings. Código: preservar arquivo e linhas.
- Cada chunk carrega document_id, página/linha, heading, hash e classe de privacidade.
- Prompt injection em documento não concede permissão para Tool Engine.

Parte 19 - Citation Engine

C

```
typedef struct {  
    int64_t document_id;  
    int page_start;  
    int page_end;  
    const char *source_name;  
    const char *heading;  
} AtenaLocator;
```

A resposta não deve inventar fonte. O Citation Engine associa trechos recuperados aos locators reais. O modelo pode sugerir em qual evidência se apoiou, mas o Core valida se a citação referenciada pertence ao conjunto de evidências realmente entregue naquele request.

Parte 20 - Tool Engine: efeitos tipados

20.1 Tool descriptor

C

```
typedef AtenaStatus (*AtenaToolInvoke)(const char *args_json, char **result_json);

typedef struct {
    const char *name;
    const char *description;
    const char *input_schema_json;
    bool requires_confirmation;
    AtenaToolInvoke invoke;
} AtenaTool;
```

20.2 Ordem correta de validação

8. O modelo propõe tool_name + arguments.
9. Parser valida JSON estrutural e limites.
10. Registry verifica se tool_name existe.
11. Request whitelist verifica atena_request_tool_allowed(request, tool_name).
12. Policy Engine verifica classe de risco, privacidade e modo.
13. Schema específico da tool valida argumentos e rejeita campos inesperados.
14. Se necessário, UI solicita confirmação humana mostrando efeito e parâmetros.
15. Somente então invoke() executa a implementação conhecida.
16. Resultado é auditado e volta como evento tipado.

Proibição estrutural — Não crie tool execute.shell(text), não concatene saída do LLM em system(), popen() ou /bin/sh -c. Ferramentas administrativas devem ser pequenas e semanticamente específicas.

Parte 21 - Primeiras tools seguras

21.1 system.info.read

C

```
AtenaStatus tool_system_info(const char *args_json, char **result_json) {
    (void)args_json;
    AtenaHardwareProfile p;
    if (atena_hw_profile(&p) != ATENA_OK) return ATENA_ERR_IO;
    /* Em produção, monte com json-c e cheque alocação. */
    return atena_json_printf(result_json,
        "{\\"ram_total\\":%llu,\\"ram_free\\":%llu,\\"cpu_threads\\":%ld}",
        p.ram_total, p.ram_free, p.cpu_threads);
}
```

21.2 brightness.set

Não escreva diretamente em /sys como root a partir do Core. O Core prepara uma ação semântica, valida 0..100 e usa um helper mínimo/Polkit ou API de desktop quando disponível. A confirmação mostra “Alterar brilho para 40%”.

Parte 22 - Agent Engine: autonomia governada

O agente é um loop, mas não um loop livre. Ele tem orçamento de passos, ferramentas permitidas, cancelamento e estado observável.

C

```
for (unsigned step = 0; step < max_steps; ++step) {
    proposal = model_generate_proposal(context);
    if (proposal.kind == FINAL_ANSWER) return emit_answer(proposal);
    if (proposal.kind != TOOL_CALL) return ATENA_ERR_PROVIDER;
    if (!registry_has(proposal.tool_name)) return ATENA_ERR_POLICY;
    if (!atena_request_tool_allowed(request, proposal.tool_name)) return ATENA_ERR_POLICY;
    if (!tool_schema_valid(proposal.tool_name, proposal.arguments)) return ATENA_ERR_INVALID;
    if (!policy_allows_tool(request, proposal.tool_name)) return ATENA_ERR_POLICY;
    if (tool_requires_confirmation(proposal.tool_name) && !confirm_with_user(...))
        return ATENA_ERR_CANCELLED;
    result = tool_invoke(...);
    context_append_tool_result(result);
}
return ATENA_ERR_TIMEOUT;
```

Falha corrigida — Schema válido não implica autorização. A whitelist da requisição é verificada antes da execução, além do Registry e Policy Engine.

Parte 23 - Auditoria e logs sem segredos

Logs são dados. Registre IDs, tipos, duração, decisão e resultado resumido; não despeje prompt, documentos, Authorization headers ou API keys no journal.

JSON

```
{
  "event": "tool.invoke",
  "request_id": 1234,
  "tool": "system.info.read",
  "policy": "allowed",
  "duration_ms": 12,
  "status": "ok"
}
```

- Redija antes de chamar o logger, não depois.
- Marque campos sensíveis no tipo de evento.
- Use IDs correlacionáveis em vez de conteúdo completo.
- Permita ao usuário exportar e limpar auditoria local.
- Rotacione logs; não transforme journal em banco infinito.

Parte 24 - systemd de usuário e sandbox coerente

INI

```
[Unit]
Description=Atena Core user service
After=graphical-session.target

[Service]
Type=simple
ExecStart=/usr/libexec/atenas/atenas-core
Restart=on-failure
RestartSec=2
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=%h/.local/share/atenas %h/.cache/atenas %h/.config/atenas
InaccessiblePaths=%h/.ssh %h/.gnupg
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6

[Install]
WantedBy=default.target
```

ProtectHome=read-only permite leitura e bloqueia escrita. Para uma Atena com RAG, isso funciona tecnicamente, mas concede leitura ampla demais. O projeto deve combinar sandbox do serviço com autorização de arquivos no aplicativo. Em uma versão mais rígida, use file chooser/portal, copie conteúdos aprovados para o espaço de dados da Atena ou crie bind mounts somente para grants persistentes.

Diagnóstico — `systemd-analyze security --user atenas-core.service` mostra a superfície do serviço. `journalctl --user -u atenas-core -f` acompanha erros de execução.

Parte 25 - IPC sem congelar a UI

A primeira versão pode usar socket Unix em \$XDG_RUNTIME_DIR/atena/core.sock. O protocolo deve ter framing explícito, limite de mensagem e autenticação do peer. JSON por linha é aceitável em protótipo, mas comprimento prefixado é mais robusto.

C

```
uint32_t len_be; /* 4 bytes, network byte order */
/* ler exatamente 4 bytes, converter ntohl, rejeitar > ATENA_MAX_IPC_MSG,
   depois ler exatamente len bytes e validar JSON. */
```

D-Bus entra depois para métodos semânticos como Ask, Cancel, ListModels, InvokeSkill. Nunca exponha ExecuteShell(string).

Parte 26 - Interface gráfica e “a sua cara”

A personalidade visual não deve contaminar o Core. A UI consome eventos e configurações, enquanto temas e identidade ficam em assets versionados.

- Defina nome, logotipo, ícones, tipografia e paleta em ui/assets e arquivos de configuração.
- Crie tokens de design: background, surface, accent, danger, success, text-primary e text-muted.
- Não codifique RGB em dezenas de widgets; use CSS/tema.
- Mostre provider/modelo ativo, modo Offline/Cloud/Híbrido e motivo de bloqueios.
- Botão Cancelar deve mapear para request_id do Core.
- Acessibilidade: navegação por teclado, contraste, foco visível, labels e leitor de tela.

26.1 Personalidade sem fine-tuning obrigatório

TEXT

```
identity/  
  persona.json  
  pedagogy.json  
  safety.json  
  regional_context.json  
  examples/
```

O Context Builder projeta somente as regras necessárias. Trocar GGUF não deve apagar a identidade da Atena. Assim você consegue personalizar “a sua cara” editando dados versionados, sem treinar modelo novamente.

Parte 27 - Como pesquisar sem depender de IA

Uma parte do trabalho é aprender a investigar bibliotecas e erros. Use uma ordem de evidência: documentação oficial, header instalado, man page, source code, issue reproduzível, fórum. Blogs entram por último.

17. Descubra a versão: `pkg-config --modversion libcurl/json-c/sqlite3`.
18. Leia o header real: `grep -R "CURLOPT_CONNECTTIMEOUT_MS" /usr/include/curl`.
19. Use man: `man 3 curl_easy_setopt`; `man 3 statvfs`; `man 3 sqlite3_prepare_v2`.
20. Compile um programa mínimo reproduzível fora da Atena.
21. Pesquise a mensagem de erro exata entre aspas, adicionando a biblioteca e versão.
22. Confirme no repositório oficial se a API mudou.
23. Registre a decisão em docs/adr se ela afetar arquitetura.

BASH

```
grep -R "json_tokenizer_new_ex" /usr/include/json-c
pkg-config --cflags --libs json-c
apt-file search json_tokenizer.h # instale apt-file se precisar
man 3 curl_easy_perform
```

Parte 28 - Pacote Debian

Quando o projeto funciona fora da ISO, empacote. A ISO deve instalar pacotes, não carregar uma cópia artesanal do source.

BASH

```
sudo apt install -y devscripts debhelper dh-make lintian
# dentro do source package:
dpkg-buildpackage -us -uc -b
lintian ../atena_*.changes
```

Artefato	Destino recomendado
atena-core	/usr/libexec/atena/atena-core
atena-cli	/usr/bin/atena
unit de usuário	/usr/lib/systemd/user/atena-core.service
schemas	/usr/share/atena/schemas/
assets	/usr/share/atena/
desktop entry	/usr/share/applications/atena.desktop
ícones	/usr/share/icons/hicolor/.../apps/atena.*

Erro corrigido — O pacote de desenvolvimento é libcurl4-openssl-dev, sem espaço depois do hífen.

Parte 29 - Integração ao Athenas OS

Atena entra na distribuição quando o pacote já instala e funciona no Mint/Debian. Na build do Athenas OS, o live-build inclui o .deb ou, preferencialmente, o repositório de pacotes do projeto. Modelos grandes não devem ser obrigatórios na ISO básica; o onboarding pode recomendar e baixar conforme hardware/licença.

- atena-core e atena-cli no perfil base do Athenas OS.
- UI opcional no desktop escolhido.
- Model Manager apresenta modelos compatíveis e tamanho antes do download.
- Modo sem modelo continua abrindo configurações, RAG lexical e diagnósticos.
- ARM64/RISC-V recebem pacotes compilados/testados por arquitetura; assets e schemas podem ser Architecture: all.

Parte 30 - CI, segurança e Definition of Done

BASH

```
# pipeline conceitual
cmake -S . -B build-gcc -DCMAKE_BUILD_TYPE=Debug
cmake --build build-gcc -j2
ctest --test-dir build-gcc --output-on-failure

CC=clang cmake -S . -B build-clang -DCMAKE_BUILD_TYPE=Debug
cmake --build build-clang -j2
ctest --test-dir build-clang --output-on-failure

cppcheck --enable=warning,style,performance,portability --error-exitcode=1 core providers tools agent
clang-tidy core/*.c -- -Iinclude
```

- GCC e Clang passam.
- ASan/UBSan passam.
- Parser JSON tem corpus de fuzz e limites.
- Nenhuma tool executa texto de shell do modelo.
- Endpoints locais não escapam do loopback quando a política é LOOPBACK_ONLY.
- Logs não contêm segredos.
- Model Manager checa espaço, hash e origem.
- RAG devolve locators reais.
- Pacote .deb instala, atualiza e remove em VM limpa.
- Atena continua útil sem modelo e sem rede.

Parte 31 - Matriz de diagnóstico por marco

Marco	Comando de verificação	Se falhar, procure primeiro
M0	<code>cmake --build build && ctest --test-dir build</code>	headers, nomes de tipos, link, ownership
M1	<code>sqlite3 DB "PRAGMA integrity_check"</code>	migração, path XDG, prepared statement
M2	<code>curl /health + atena ask</code>	endpoint, firewall local, JSON, timeout, callback
M3	<code>atena model probe</code>	statvfs, hash, RAM, formato GGUF
M4	<code>sqlite3 DB "SELECT count(*) FROM chunks_fts"</code>	triggers FTS, extractor, query MATCH
M5	teste de tool não autorizada	whitelist, policy, schema, confirmação
M6	<code>systemctl --user status atena-core</code>	unit, paths, sandbox, IPC permissions
M7	<code>dpkg -i / apt install local</code>	dependências, file destinations, maintainer scripts
M8	CI + QEMU/hardware	endian/ISA, libs disponíveis, desempenho real

31.1 Receita universal para um erro de compilação

24. Leia a primeira mensagem de erro, não as últimas 30 em cascata.
25. Copie arquivo:linha e abra no Geany.
26. Veja o tipo/símbolo citado com `grep -R`.
27. Se “undeclared/unknown type”, é declaração/include/nome.
28. Se “undefined reference”, é link/implementação.
29. Se warning de conversão, investigue tamanho/sinal antes de cast.
30. Rode `build verbose` para ver a linha exata de gcc/ld.
31. Crie teste mínimo se a dúvida for de biblioteca externa.
32. Só depois pesquise na web; inclua versão e erro exato.

Parte 32 - Personalização técnica avançada

Depois da baseline estável, a sua versão pode divergir em módulos, não em gambiarras. Toda personalização deve responder: qual contrato toca? como testa? como desativa? como migra?

Quero mudar...	Lugar certo
personalidade/voz pedagógica	identity/*.json e policy pack
visual/cores	ui/assets e tema CSS
modelo padrão	config + Model Manager
provider remoto	providers/ novo adapter
nova automação	tools/ + schema + policy
agente especializado	skills/agent manifest + tools permitidas
conhecimento regional	RAG/base de conhecimento com fontes
comportamento por hardware	Compatibility Engine + perfis
integração com app do Athenas	D-Bus/portal/Skill API

Parte 33 - Projeto final: sequência de implementação do zero ao avançado

33. Instale toolchain no Linux Mint e valide GCC/CMake.
34. Crie repositório e commit inicial.
35. Implemente status/request/event/provider e testes.
36. Implemente provider mock e CLI.
37. Adicione Policy Engine e allowed_tools.
38. Adicione SQLite com migrações e retenção.
39. Compile llama.cpp e valide /health e /v1/models.
40. Implemente provider HTTP com URL parser, timeouts, streaming e cancelamento.
41. Implemente Model Manager com statvfs, hash, catálogo e benchmark.
42. Implemente Hardware Profiler e Compatibility Engine.
43. Implemente Provider Router e Model Router.
44. Implemente ingestão RAG, chunks, FTS5 external content, triggers e citações.
45. Implemente Tool Registry, schemas, whitelist e primeira tool read-only.
46. Implemente Agent Engine com max_steps, confirmação e auditoria.
47. Mova o Core para serviço systemd de usuário e IPC.
48. Crie UI que consome o mesmo protocolo e nunca duplica política.
49. Adicione identidade/persona e personalização visual.
50. Empacote em .deb, rode Lintian, teste upgrade/removal.
51. Integre o pacote ao Athenas OS e teste em ISO/VM.
52. Abra matriz ARM64/RISC-V somente depois da baseline amd64 estar reproduzível.

Quando considerar “Atena criada” — Quando trocar o modelo/provider sem reescrever o Core; funcionar offline em modo degradado; responder com RAG e fontes; executar tools autorizadas; cancelar requests; persistir dados de forma controlada; instalar como pacote; e explicar suas decisões principais ao usuário.

Apêndice A - Erros do PDF anterior e correções incorporadas

Problema detectado	Status nesta edição
JSON mock inválido	Corrigido com escape C correto.
ATENA ERR_INVALID	Corrigido para ATENA_ERR_INVALID.
ProviderTable/VTable	Unificado em AtenaProviderVTable.
CURLOPT_* MS	Corrigido para CURLOPT_CONNECTTIMEOUT_MS / CURLOPT_TIMEOUT_MS.
json_object_new double	Corrigido para json_object_new_double.
libcurl4- openssl-dev	Corrigido para libcurl4-openssl-dev.
conversation id	Corrigido para conversation_id.
chunks_c e FTS5 inconsistente	Corrigido com chunks + external content + triggers.
hardening HOME x RAG	Reformulado: ProtectHome=read-only é leitura ampla; grants e paths sensíveis são controlados.
endpoint sem validação	Parser libcurl URL + host loopback exato.
tool só com schema	Whitelist + Registry + Policy + schema + confirmação.
HTTP não cancelável	XFERINFO callback + separação da UI; multi/event loop como evolução.
JSON sem limites	tokener depth 32, tamanho máximo, checagem de tipos/NULL.
Model Manager sem disco	statvfs antes do download + margem.

Apêndice B - Comandos de recuperação rápida

BASH

```
# limpar e reconstruir
rm -rf build
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure

# ver dependências
pkg-config --cflags --libs json-c sqlite3 libcurl
ldd build/aten-cli

# símbolos
nm -C build/libaten-core.a | less

# debugger
gdb --args ./build/aten-cli ask "teste"

# sistema
systemctl --user daemon-reload
systemctl --user restart aten-core
systemctl --user status aten-core
journalctl --user -u aten-core -n 100 --no-pager

# banco
sqlite3 ~/.local/share/aten/aten.db "PRAGMA integrity_check;"

# runtime local
curl -s http://127.0.0.1:18080/health | jq .
```

Apêndice C - Glossário de programação aplicado

Termo	Significado prático na Atena
Header .h	contrato visível para outros módulos.
Source .c	implementação do contrato.
Compilador	transforma C em objeto/código de máquina.
Linker	junta objetos e bibliotecas.
ABI	contrato binário entre componentes compilados.
Pointer	endereço de um objeto; exige ciclo de vida claro.
Ownership	quem cria, usa e libera memória/recurso.
Callback	função fornecida para receber eventos.
VTable	tabela de funções que permite providers substituíveis.
Schema	regras estruturais para dados JSON.
Prepared statement	SQL compilado com valores bindados, evitando concatenação insegura.
FTS5	índice de busca textual do SQLite.
Provider	adapter para runtime/modelo local ou remoto.
Router	componente que escolhe candidato já permitido.
Tool	efeito tipado e autorizado.
Agent	loop governado que pode propor tools.
IPC	comunicação entre processos.
Polkit	autorização de operações privilegiadas sem root genérico.
Sanitizer	instrumentação que detecta erros de memória/UB.
Fuzzing	geração de entradas inesperadas para testar parsers.

Conclusão - o que este livro entrega

Atena não termina quando o primeiro modelo responde. Ela termina uma primeira geração quando possui uma plataforma própria: contratos, política, providers, memória, conhecimento, tools, agentes, auditoria, interface e integração ao Linux. O modelo continua sendo uma peça substituível. Essa é a diferença entre construir um chatbot e construir uma capacidade nativa de sistema operacional.

A melhor forma de usar este livro é implementar um marco por vez e manter uma branch “main” sempre compilável. Quando um capítulo divergir da versão atual de uma biblioteca externa, use o método de pesquisa descrito aqui, atualize somente o adapter correspondente e registre a decisão. A arquitetura da Atena foi desenhada precisamente para que o mundo externo possa mudar sem obrigar o Core a se desfazer.